Module 1: Java

Java Crash Course

Java is to JavaScript what Pine is to Pineapple.

0.

Communicating Code

"The single biggest problem in communication is the illusion that it has taken place." *~George Bernard Shaw*

The Psuedocode Problem

% Operator	Remainder: C++, Java
	`7 % 3 == 1`
	`-7 % 3 == -1`
		Modulo: Python, Ruby, Perl
		`7 % 3 == 1`
		`-7 % 3 == 2`

Indexing	Zero: C++, Java `first = array[0]`	One: Matlab, Lua `first = array[1]`
Ranges	Inclusive: `1..10` (Ruby) `1...10` (Swift) `range(1, 10)` (PHP)	Exclusive: `1...10` (Ruby) `1..<10` (Swift) `range(1, 10)` (Python)

Syntax Across Languages

Watch for differences across languages!

especially with terminology…

```java
// A "best practice" class
public class Vector {

    private int x;
    private int y;

    public Vector(int x, int y) {
        this.x = x;
        this.y = y;
    }

    public double getLength() {
        return Math.sqrt(x * x + y * y);
    }

}
```

```java
// A "good enough" class
public class Vector {
  private int x, y; // get, set
  public Vector(int x, int y) {…}
  public double getLength() {
    return Math.sqrt(x*x + y*y);
  }
}

// A "bare minimum" method
// class Vector(int x, int y)
public double getLength() {
  return Math.sqrt(x*x + y*y);
}
```

Example `Vector` class (Java)

Examples are just that…
Examples!

Too little context and real-world utility is unclear.

Too much context and the idea itself is unclear.

1.

The Java Ecosystem

It's alive, it's alive!

The Java Ecosystem

Java and the JVM

- Strong, statically typed language
- Designed for OOP, inspired by C++
- Compiled to Java Bytecode (.class)
  - Interpreted by the Java Virtual Machine (JVM)
  - “Write once, run anywhere”
  - Just-In-Time (JIT) compilation
- JVM Langs: Kotlin, Scala, Clojure

Notable Tools/Technologies

- JDK: Java Development Kit
  - javac (Java Compiler)
- Build Systems: Gradle, Maven
- Libraries: Spring, JUnit, Apache Commons, Google Guava, Lombok
- IDE: IntelliJ IDEA, Eclipse
- Documentation: JavaDocs

2.

Language Fundamentals

Expressions, Statements, Objects, & Exceptions: oh my!

```java
public class Main {                                  // main class:
                                                     // Main: conventional name

  public static void main(String[] args) {           // main method:
                                                     // public: publicly visible
                                                     // static: no object instance
                                                     // void: no return value
                                                     // main: required name (JVM)
                                                     // String[] args: args array

    System.out.println("Hello, World!");             // System.out: standard out
                                                     // println: print with newline
                                                     // "Hello World!": message
  }

}
```

Java: Hello, World!

```java
public class Main {
  public static void main(String[] args) {
    for (int i = 1; i <= 100; i++) {
      if (i % 3 == 0 && i % 5 == 0) {
        System.out.println("FizzBuzz");
      } else if (i % 3 == 0) {
        System.out.println("Fizz");
      } else if (i % 5 == 0) {
        System.out.println("Buzz");
      } else {
        System.out.println(i);
      }
    }
  }
}
```

Java: FizzBuzz

```java
int integer = 1;
Integer boxed = integer;                    // autoboxed to Integer object
assert integer == boxed;                    // question: How about two Integers?

Person alice = new Person("Alice");         // new: Creates a Person instance
Person aliceAgain = alice;                  // references behave like pointers
assert alice == aliceAgain;                 // same reference, same instance!

Person bob = new Person("Bob");
Person bobThe2nd = new Person("Bob");
assert bob != bobThe2nd;                    // different instance!
assert bob.equals(bobThe2nd);               // given Person implements equals!

Person clair = null;                        // caution: References can be null!
assert clair.equals(null);                  // throws NullPointerException!
assert clair == null;
```

Java: Primitive vs Reference Types

```java
String string = …;

try {
  int n = Integer.parseInt(string);
  //success!
} catch (NumberFormatException e) {
  //failure
}

//concern: this compiles???
Integer.parseInt(string);
//java.lang.NumberFormatException
```

```java
String string = …;

try {
  var tokens = Lexer.lex(string);
  //success!
} catch (LexException e) {
  //failure
}

//advantage: this errors!
Lexer.lex(string);
//(no runtime error - checked!)

//concern: painful with lambdas
Stream.of(string).map(Lexer::lex);
```

Java: Unchecked vs Checked Exceptions

3.

Modern Java

Modern problems require modern solutions.

```java
List<String> strings = List.of(
  "123", "-456", "789"
);

var nums = new ArrayList<Integer>();
for (var str : strings) {
    var num = Integer.parseInt(str);
    if (num > 0) {
      nums.add(num);
    }
}
assert nums == List.of(123, 789);

//concern: imperative - operations
//are clearer, but not the effect
//concern: nums is mutable!
```

```java
List<String> strings = List.of(
  "123", "-456", "789"
);

var nums = strings.stream()
  .map(s -> Integer.parseInt(s))
  .filter(n -> n > 0)
  .toList();
assert nums == List.of(123, 789);

//advantage: declarative - overall
//effect of computation is clearer
//advantage: nums is immutable
//concern: stream/toList overhead
//concern: painful with lambdas
Stream.of(string).map(Lexer::lex);
```

Java 8: Lambda Expressions

```java
String message = "Hello, World!";

String declared;
String nullable = null;

class Name {
  private String field = "value";
}

List<String> msgs = new ArrayList<>();
msgs = new LinkedList<String>();


List<Integer> ints = List.of(1, 2, 3);
List<Number> nums = List.of(1, 2.0, 3);
```

```java
var message = "Hello, World!";

var declared
var nullable = null

class Name {
  private var field = "value";
}

var msgs = new ArrayList<String>();
msgs = new LinkedList<String>();
// typeof(msgs) == ArrayList<String>

var ints = List.of(1, 2, 3);
var nums = List.of(1, 2.0, 3);
// List<Number & Comparable<? extends …
```

Java 10: Local-Variable Type Inference

```java
enum Direction { NORTH, EAST, … };
Direction direction = …;

Vector delta;
switch (direction) {
  case NORTH:
    delta = new Vector(0, 1);
    break;
  case EAST:
    delta = new Vector(1, 0);
    break;
  case SOUTH: …
  case WEST: …
  default: // now what?
}
```

```java
enum Direction { NORTH, EAST, … };
Direction direction = …;

Vector delta = switch (direction) {
  case NORTH → new Vector(0, 1);
  case EAST → new Vector(1, 0);
  case SOUTH → new Vector(0, -1);
  case WEST → {
    var delta = new Vector(-1, 0);
    yield delta;
  }
  // compiler error on missing case!
}
```

Java 12-14: Switch Expressions

4.

Standard Library

No joke here, Java's standard library is actually pretty lit.

Java Standard Library

Absolute Must Know

- `java.lang`
  - `Object`, `String`, `Integer`, etc.
  - `Comparable<T>`, `Iterable<T>`
  - `Exception`, `RuntimeException`
- `java.util`
  - Collections (`List`, `Map`, `Set`)
  - `Objects`, `Arrays`, `Collections`
  - `Optional<T>` (type-safe `null`)
  - `Date`, `Random`, `Scanner`, etc.

Other Notable Packages

- `java.awt`
- `java.io`/`java.nio`
- `java.math`
- `java.net`
- `java.sql`/`javax.sql`
- `java.time`

JavaDocs

- `Integer.parseInt`

Module 1: Java

Java Crash Course

Java is to JavaScript what Pine is to Pineapple.

Slides Carnival

Free templates for all your presentation needs

For PowerPoint and Google Slides

100% free for personal or commercial use

Ready to use, professional and customizable

Blow your audience away with attractive visuals